Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Янчевская А.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 30.10.26

Москва, 2026

Постановка задачи

Вариант 7.

Два человека играют в кости. Правила игры следующие: каждый игрок делает бросок 2-ух костей K раз; побеждает тот, кто выбросил суммарно большее количество очков. Задача программы экспериментально определить шансы на победу каждого из игроков. На вход программе подается K, какой сейчас тур, сколько очков суммарно у каждого из игроков и количество экспериментов, которые должна произвести программа

Общий метод и алгоритм решения

Пользователь задает параметры игры: общее количество раундов k, номер текущего раунда curr_turn, начальный счет между игроками score1 score2 и количество экспериментов experiments. Каждый эксперимент начинается с указанных начальных очков, после чего моделируются оставшиеся раунды. В каждом раунде оба игрока бросают по два кубика, при равенстве очков раунда баллы не начисляются. После завершения игры сравниваются итоговые результаты игроков, и победа засчитывается тому, у кого счет больше.

Для оценки производительности реализованы две версии программы: последовательная и параллельная. Каждый поток независимо подсчитывает победы, а после завершения работы результаты объединяются. Для обеих версий также измеряется время выполнения. На основе полученных значений рассчитываются ускорение параллельной программы и эффективность использования потоков.

Число потоков	Время последовательной версии, мс	Время параллельной версии, мс	Ускорение	Эффективность
1	2061	2091	0,99	98,6%
2	2079	1998	1,04	52,0%
4	2172	1406	1,54	38,6%
8	1500	766	1,96	24,5%
12	1498	561	2,67	22,3%
16	1510	473	3,19	20,0%
64	1515	271	5,59	8,7%
128	1483	287	5,17	4,0%
256	1499	261	5,74	2,2%
512	1513	307	4,93	1,0%
1024	1508	339	4,45	0,4%

read() - чтение входных данных из стандартного потока ввода

write() - вывод результатов и сообщений об ошибках

open() - открытие /dev/urandom для получения начального значения генератора

close() - закрытие открытого файлового дескриптора

clock_gettime() - получение времени для измерения производительности

pthread_create() - создание нового потока

pthread_join() - ожидание завершения потока

Код программы

lab.h

```
#ifndef LAB_H
#define LAB_H

#include <fcntl.h>
#include <pthread.h>
#include <time.h>
#include <unistd.h>

typedef struct {
  int k;
  int curr_turn;
  int score1;
  int score2;
  long experiments;
} ExperimentData;

typedef struct {
  ExperimentData *data;
  long experiments;
  long wins1;
  long wins2;
  unsigned int seed;
} ThreadData;

void print_error(void);
int read_num(long *num);
int get_seed(unsigned int *seed);
int roll_dice(unsigned int *seed);
void play_round(unsigned int *seed, int *score1, int *score2);

void run_experiment(ExperimentData *data, unsigned int *seed, int *score1,
                    int *score2);

int run_seq(ExperimentData *data, long *wins1, long *wins2);
int run_parallel(ExperimentData *data, int thread_count, long *wins1,
                 long *wins2);

double get_time(struct timespec start, struct timespec end);

void *thread_worker(void *arg);

#endif
```

main.c

```
#include "lab.h"

void print_error(void) { write(2, "error\n", 6); }

int num_to_str(long num, char *str) {
  int i = 0;
  if (num == 0) {
    str[i++] = '0';
  }
  while (num > 0) {
    str[i++] = num % 10 + '0';
    num /= 10;
  }
  str[i] = '\0';
  for (int j = 0; j < i / 2; j++) {
    char temp = str[j];
    str[j] = str[i - j - 1];
```

```
    str[i - j - 1] = temp;
  }
  return i;
}

int read_num(long *num) {
  char buff[32];
  long value = 0;
  int i = 0;
  int bytes = read(0, buff, sizeof(buff));
  if (bytes <= 0) {
    return -1;
  }
  while (i < bytes && buff[i] >= '0' && buff[i] <= '9') {
    value = value * 10 + buff[i] - '0';
    i++;
  }
  if (i == 0 || (i < bytes && buff[i] != '\n')) {
    return -1;
  }
  *num = value;
  return 0;
}

int get_seed(unsigned int *seed) {
  int fd = open("/dev/urandom", O_RDONLY);
  if (fd < 0) {
    return -1;
  }
  int bytes = read(fd, seed, sizeof(*seed));
  close(fd);
  if (bytes != sizeof(*seed)) {
    return -1;
  }
  return 0;
}

int roll_dice(unsigned int *seed) {
  *seed = *seed * 1103515245 + 12345;
  return (*seed % 6) + 1;
}

void play_round(unsigned int *seed, int *score1, int *score2) {
  int p1 = roll_dice(seed) + roll_dice(seed);
  int p2 = roll_dice(seed) + roll_dice(seed);
  if (p1 > p2) {
    (*score1)++;
  } else if (p2 > p1) {
    (*score2)++;
  }
}

void run_experiment(ExperimentData *data, unsigned int *seed, int *score1,
                    int *score2) {
  *score1 = data->score1;
  *score2 = data->score2;
  int remain_turns = data->k - data->curr_turn;
  for (int i = 0; i < remain_turns; i++) {
    play_round(seed, score1, score2);
  }
}

int run_seq(ExperimentData *data, long *wins1, long *wins2) {
  unsigned int seed;
  if (get_seed(&seed) != 0) {
    return -1;
```

```
  }
  *wins1 = 0;
  *wins2 = 0;
  for (long i = 0; i < data->experiments; i++) {
    int score1, score2;
    run_experiment(data, &seed, &score1, &score2);
    if (score1 > score2) {
      (*wins1)++;
    } else if (score2 > score1) {
      (*wins2)++;
    }
  }
  return 0;
}

double get_time(struct timespec start, struct timespec end) {
  return (end.tv_sec - start.tv_sec) * 1000.0 +
         (end.tv_nsec - start.tv_nsec) / 1000000.0;
}

int main(int argc, char *argv[]) {
  if (argc != 2) {
    print_error();
    return 1;
  }
  long thread_value = 0;
  int i = 0;
  while (argv[1][i] >= '0' && argv[1][i] <= '9') {
    thread_value = thread_value * 10 + argv[1][i] - '0';
    i++;
  }
  if (i == 0 || argv[1][i] != '\0' || thread_value <= 0) {
    print_error();
    return 1;
  }
  int thread_count = (int)thread_value;
  ExperimentData data;
  long value;
  if (read_num(&value) != 0 || value <= 0) {
    print_error();
    return 1;
  }
  data.k = (int)value;
  if (read_num(&value) != 0 || value < 0 || value > data.k) {
    print_error();
    return 1;
  }
  data.curr_turn = (int)value;
  if (read_num(&value) != 0) {
    print_error();
    return 1;
  }
  data.score1 = (int)value;
  if (read_num(&value) != 0) {
    print_error();
    return 1;
  }
  data.score2 = (int)value;
  if (read_num(&value) != 0 || value <= 0) {
    print_error();
    return 1;
  }
  data.experiments = value;
  long wins1, wins2;
  struct timespec start, end;
  clock_gettime(CLOCK_MONOTONIC, &start);
```

```
  if (run_seq(&data, &wins1, &wins2) != 0) {
    print_error();
    return 1;
  }
  clock_gettime(CLOCK_MONOTONIC, &end);
  double seq_time = get_time(start, end);
  long parallel_wins1, parallel_wins2;
  clock_gettime(CLOCK_MONOTONIC, &start);
  if (run_parallel(&data, thread_count, &parallel_wins1, &parallel_wins2) !=
      0) {
    print_error();
    return 1;
  }
  clock_gettime(CLOCK_MONOTONIC, &end);
  double parallel_time = get_time(start, end);
  char buff[64];
  int len;

  len = num_to_str(wins1, buff);
  write(1, "Seq1: ", 6);
  write(1, buff, len);
  write(1, "\n", 1);

  len = num_to_str(wins2, buff);
  write(1, "Seq2: ", 6);
  write(1, buff, len);
  write(1, "\n", 1);

  len = num_to_str(parallel_wins1, buff);
  write(1, "Par-lel1: ", 10);
  write(1, buff, len);
  write(1, "\n", 1);

  len = num_to_str(parallel_wins2, buff);
  write(1, "Par-lel2: ", 10);
  write(1, buff, len);
  write(1, "\n", 1);

  len = num_to_str((long)seq_time, buff);
  write(1, "Seq time: ", 10);
  write(1, buff, len);
  write(1, "\n", 1);

  len = num_to_str((long)parallel_time, buff);
  write(1, "Par-lel time: ", 14);
  write(1, buff, len);
  write(1, "\n", 1);

  return 0;
}
```

parallel.c

```
#include "lab.h"

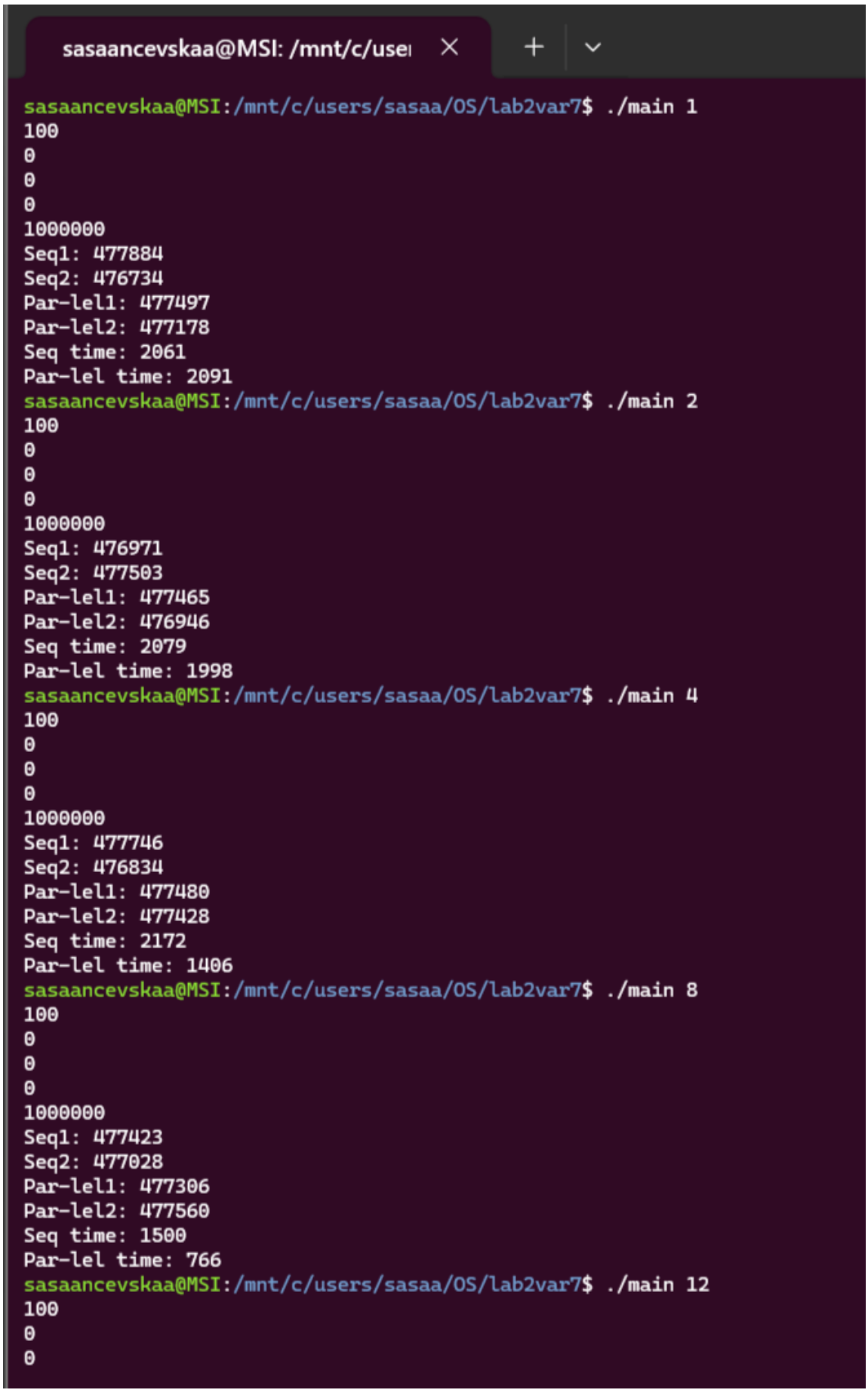
void *thread_worker(void *arg) {
  ThreadData *thread_data = (ThreadData *)arg;
  thread_data->wins1 = 0;
  thread_data->wins2 = 0;
  for (long i = 0; i < thread_data->experiments; i++) {
    int score1, score2;
    run_experiment(thread_data->data, &thread_data->seed, &score1, &score2);
    if (score1 > score2) {
      thread_data->wins1++;
    } else if (score2 > score1) {
      thread_data->wins2++;
```

```
    }
  }
  return NULL;
}

int run_parallel(ExperimentData *data, int thread_count, long *wins1,
                 long *wins2) {
  pthread_t threads[thread_count];
  ThreadData thread_data[thread_count];
  long base = data->experiments / thread_count;
  long rem = data->experiments % thread_count;
  *wins1 = 0;
  *wins2 = 0;
  for (int i = 0; i < thread_count; i++) {
    thread_data[i].data = data;
    thread_data[i].experiments = base + (i < rem);
    if (get_seed(&thread_data[i].seed) != 0) {
      return -1;
    }
  }
  int created = 0;
  for (int i = 0; i < thread_count; i++) {
    if (pthread_create(&threads[i], NULL, thread_worker, &thread_data[i]) !=
        0) {
      for (int j = 0; j < created; j++) {
        pthread_join(threads[j], NULL);
      }
      return -1;
    }
    created++;
  }
  for (int i = 0; i < thread_count; i++) {
    pthread_join(threads[i], NULL);
    *wins1 += thread_data[i].wins1;
    *wins2 += thread_data[i].wins2;
  }
  return 0;
}
```

Протокол работы программы

Тестирование:



```
sasaancevskaa@MSI: /mnt/c/use
100
0
0
0
1000000
Seq1: 477441
Seq2: 477011
Par-lel1: 477715
Par-lel2: 477058
Seq time: 1498
Par-lel time: 561
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 16
100
0
0
0
1000000
Seq1: 477918
Seq2: 476512
Par-lel1: 477218
Par-lel2: 477236
Seq time: 1510
Par-lel time: 473
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 64
100
0
0
0
1000000
Seq1: 477840
Seq2: 476546
Par-lel1: 478432
Par-lel2: 476344
Seq time: 1515
Par-lel time: 271
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 128
100
0
0
0
1000000
Seq1: 477625
Seq2: 476846
Par-lel1: 477302
Par-lel2: 477206
Seq time: 1483
Par-lel time: 287
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 256
100
0
0
0
```

```
sasaancevskaa@MSI: /mnt/c/use
Par-lel time: 287
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 256
100
0
0
0
1000000
Seq1: 477413
Seq2: 477482
Par-lel1: 477229
Par-lel2: 476828
Seq time: 1499
Par-lel time: 261
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 512
100
0
0
0
1000000
Seq1: 477353
Seq2: 477145
Par-lel1: 477049
Par-lel2: 477458
Seq time: 1513
Par-lel time: 307
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$ ./main 1024
100
0
0
0
1000000
Seq1: 477013
Seq2: 477305
Par-lel1: 477447
Par-lel2: 477202
Seq time: 1508
Par-lel time: 339
sasaancevskaa@MSI:/mnt/c/users/sasaa/OS/lab2var7$
```

Вывод

В ходе лабораторной работы были реализованы последовательная и параллельная версии программы для моделирования игры двух игроков. Сравнение времени выполнения показало, что использование нескольких потоков позволяет ускорить вычисления, однако увеличение их количества не всегда приводит к дальнейшему росту производительности. Также были рассчитаны ускорение и эффективность параллельной программы для разных значений числа потоков. В результате удалось изучить принципы многопоточного программирования и оценить влияние количества потоков на скорость выполнения программы.